

L39: MDP Framing for Simulation Control

Simulation Without a Black Box

Outline

RL Motivation

MDP Components

Mapping Queue to MDP

Episodes vs. Continuing Tasks

Bellman Equation

Tabular Setting

SimPy as an MDP Engine

Key Takeaways

Why Reinforcement Learning for Simulation?

Simulation Studies (Classical)

Question: How good is design x ?

Answer: Run replications, estimate $\mathbb{E}[f(x)]$.

The policy is fixed before the simulation starts.

RL / MDP Framing

Question: What should the system *do next* given its current state?

Answer: Learn a policy $\pi(a|s)$ that maps observed states to actions to maximise cumulative reward.

The policy *adapts* to state during the simulation.

Motivating example:

A clinic must decide each morning how many staff to schedule based on forecasted demand and current queue length.

- State: queue length, day of week, occupancy
- Action: staff level (1–5 nurses)
- Reward: $-(\text{wait time} + \text{idle cost})$

No closed-form policy exists. RL learns one.

MDP Components: (S, A, P, R, γ)

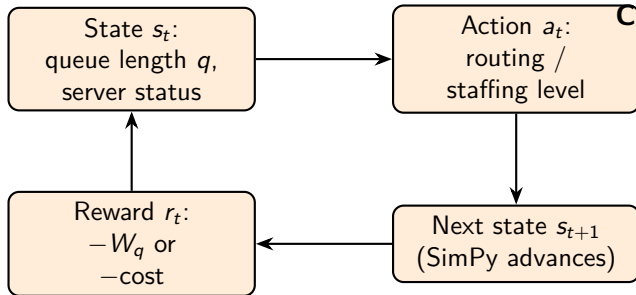
Symbol	Name	Meaning
S	State space	All observable configurations of the system
A	Action space	Decisions available to the agent
$P(s' s, a)$	Transition	Probability of next state given current state and action
$R(s, a, s')$	Reward	Immediate signal received after taking action a in state s
$\gamma \in [0, 1)$	Discount	Weights future rewards; $\gamma \rightarrow 0 = \text{myopic}$, $\gamma \rightarrow 1 = \text{long-horizon}$

Markov Property

$$P(s_{t+1}|s_t, a_t, s_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

The future is independent of history *given* the current state.

Mapping a Queuing System to an MDP



Common state encodings:

- Queue length (discretized to bins)
- Number of busy servers / utilisation
- Time of day (for non-stationary arrivals)
- Inventory level

Common action spaces:

- Routing: send arriving customer to server 1 or 2
- Staffing: open/close a server
- Inventory: order quantity or order-up-to level

Episode vs. Continuing Task

Episodic Task

Simulation runs for a fixed horizon T then resets.

Example: simulate one 8-hour shift. At the end, reset the queue and start a new shift.

Return: $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$

Continuing Task

No natural episode boundary; simulation runs forever.

Example: a hospital that never closes.

Use average reward formulation:

$$\rho^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=0}^T r_t \right]$$

Warning

Applying a discounted episodic RL algorithm to a naturally continuing task without careful truncation introduces bias — check your episode definition.

Bellman Optimality Equation

State-Value Bellman Equation

$$V^*(s) = \max_{a \in \mathcal{A}} \left[\sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \right]$$

Action-Value (Q) Bellman Equation

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

Interpretation:

The value of a state equals the immediate reward plus the discounted value of the best next state.

Why it matters:

Q-learning, DQN, and policy gradient methods all arise from approximating or solving a version of this equation.

Tabular MDP: Finite State and Action Spaces

When tabular methods work:

- $|\mathcal{S}|$ and $|\mathcal{A}|$ are small enough to enumerate
- E.g., queue length $q \in \{0, 1, \dots, 20\}$ and staffing $c \in \{1, 2, 3\} \Rightarrow 63$ state-action pairs

Value iteration (exact):

1. Initialise $V(s) = 0$ for all s
2. Repeatedly apply Bellman backup until convergence
3. Extract policy: $\pi(s) = \arg \max_a Q(s, a)$

Convergence Guarantee

Value iteration converges to V^* in the ℓ_∞ norm at rate γ^k after k sweeps — guaranteed for finite MDP with $\gamma < 1$.

Limitation

When $|\mathcal{S}|$ grows (multi-server, multi-product) the table becomes intractable. Function approximation (deep RL) is then required.

SimPy as an MDP Engine

SimPy processes as state transitions:

1. Agent calls `env.step(action)`
2. SimPy advances time until the next decision epoch (e.g., customer arrival)
3. Observe new queue length $\Rightarrow$ new state s'
4. Compute reward $r = -\text{wait}$
5. Return (s', r, done) to agent

```
class QueueMDP:
    def reset(self):
        self.env =
             $\hookrightarrow$  simpy.Environment()
        self.q = 0
        return self._obs()

    def step(self, action):
        # action: route to server 1
         $\hookrightarrow$  or 2
        self._dispatch(action)
        self.env.run(until=self._next_event())
         $\hookrightarrow$  xt_epoch()
        obs = self._obs()
        rew = -self.last_wait
        done = self.env.now >= 480
        return obs, rew, done, {}
```

Key Takeaways

L39 Summary

1. RL extends simulation from passive evaluation to active control — the policy adapts to observed system state.
2. An MDP is defined by (S, A, P, R, γ) ; the Markov property is required.
3. Queue control maps naturally: state = queue/server status, action = routing/staffing, reward = $-$ wait or $-$ cost.
4. Episodic and continuing task formulations require different return definitions — match the formulation to the physical system.
5. SimPy becomes an MDP engine by exposing `reset()` and `step(action)` interfaces.

Next: L40 — Tabular Q-learning for queuing control.